计算机体系结构
实验 3
异常与中断的添加

鲁昕宁　2414015

2026 年 11 月 10 日

GitHub 仓库　SidneyLu/Computer-Architecture-NKU-CS/tree/master/lab3

班号　0964

授课教师　董前琨

1 实验目标

本实验接着实验 2 的五级流水线，完成《CPU 设计实战》第 7 章两个实践任务。CPU 仍是 `mycpu_verify/rtl/myCPU`。

1. 增加 `MFC0`、`MTC0`、`ERET` 和 `SYSCALL`，以及 CP0 的 Status、Cause、EPC，使 `func_lab8` 的 69 个功能点与参考轨迹一致。
2. 增加 Break、地址错、整数溢出和保留指令，以及 Count、Compare、BadVAddr。定时器中断固定在硬件中断 5，即 Cause 的 IP7。硬件中断 0～5 对应 IP7～IP2，软件中断对应 IP1、IP0。`func_lab9` 共 94 个功能点。

前递、load-use 停顿、HI/LO 和延迟槽沿用实验 1、实验 2。实验箱未接，比特流没有下载。`func_lab8`、`func_lab9` 的轨迹对比之外，另有一段定向程序把 SYSCALL、其余异常和三种中断的 Cause、EPC 记下来并画出波形。记忆游戏用官方二进制在仿真里走完一轮。

2 实验原理

异常和中断都在写回级写入 CP0。写回之前的三级只负责判定，并把 BD、异常有效位、ExcCode、CP0 操作和 BadVAddr 随指令往后传。中断的 ExcCode 是 0，和“没有异常”无法用编码本身区分，所以总线上单独有一位 `ex`。

流水级	本级负责的事
译码	`SYSCALL`、`BREAK`、保留指令、取指地址错；标记延迟槽；给 `ADD`/`ADDI`/`SUB` 带上溢出检查
执行	有符号溢出、读写地址错；置 `ex_cancel`，关掉数据 RAM 写使能
访存	把异常信息、`rt` 和 BadVAddr 交给写回；地址错的 load 不再产生写回数据
写回	更新 CP0，响应中断，清空流水线；`MFC0` 在这里读出 CP0

2.1 CP0

CP0 是写回级里的一组寄存器，布局按参考 CPU 的 GS132，因为 trace 比较的是 `MFC0` 写出的 32 位，处理程序后来做的掩码不影响这次比较。

复位时 Status 为 `0x10400004`：CU0（位 28）、BEV（位 22）、ERL（位 2）为 1。Cause 的 ExcCode 复位为 `0x1f`，其余位为 0。BEV 为 1 时异常入口是 `0xbfc00380`，否则是 `0x80000180`。测试在进入异常前执行 `mtc0`，把 Status 写成 `0x00400000`。这次写入留下 BEV，清掉 ERL、EXL 和 IE，入口仍是 `0xbfc00380`。

`MTC0` 写 Status 时只改这些位：CU0、BEV、NMI、IM7～IM0、KSU、ERL、EXL、IE。异常写入 CP0 时把 EXL（位 1）置 1，因此 `SYSCALL` 之后 Status

变成 0x00400002。ERET 总是清 ERL；ERL 为 0 时同时清 EXL。同一拍里异常优先于 ERET，ERET 又优先于 MTC0 对 EXL、ERL 的修改。

EPC 和 Cause.BD 只在把异常写入 CP0、且当时 EXL 为 0 时更新。发生异常的指令带 BD 时，EPC 等于它的 PC 减 4，否则 EPC 就是它的 PC。EXL 已经为 1 时再进入异常，只改 ExcCode，EPC 保持不变。处理程序里的 MTC0 写 EPC 本身不是异常，用来把返回地址改成加 4 或加 8 之后的值。

Cause 按 MIPS 的域拼出来：位 31 是 BD，位 30 是 TI，位 15～8 是 IP7～IP0，位 6～2 是 ExcCode。CE、DC、IV 等本实验用不到的域保持 0。MFC0 可读的号是 8、9、11、12、13、14，对应 BadVAddr、Count、Compare、Status、Cause、EPC，其它号返回 0。

来源	ExcCode
中断	0
取指地址错、load 地址错（AdEL）	4
store 地址错（AdES）	5
SYSCALL	8
BREAK	9
保留指令	10
ADD/ADDI/SUB 溢出	12

2.2 各类异常怎么判定

指令编码和 PC 在译码级已经齐全，下面几类在这里确定。

- SYSCALL、BREAK 是特殊指令功能码 0x0c、0x0d。它们不写通用寄存器。
- 保留指令：操作码不属于实验 2 已经实现的集合，也不是 MFC0、MTC0、ERET。取指地址错优先于保留指令，避免把一个错地址上读来的字再当成非法操作码。
- 取指地址错：该指令的 PC 最低两位不是 0。BadVAddr 记这个 PC。同时把访存操作、乘除操作和寄存器写使能清掉，分支也不采纳，这样错地址上读到的编码不会去写 HI/LO 或改 PC。

执行级能看到 ALU 结果之后再补两类。译码级已经判定为异常的指令保持原来的 ExcCode。

- 溢出只检查 ADD、ADDI、SUB。加法在两个操作数符号相同、结果符号与它们不同时溢出。减法在两个操作数符号不同、结果符号与被减数不同时溢出。ADDU、ADDIU、SUBU 不检查。溢出的算术指令译码时本来会写寄存器，写回级看到异常后把 rf_we 拉低，目的寄存器保持进入异常之前的值。
- LW、SW 在地址最低两位非 0 时分别是 AdEL、AdES。LH、LHU、SH 只看最低一位。LB、LBU、SB 以及 LWL/LWR/SWL/SWR 允许任意字节对齐。访存地址错的 BadVAddr 是 ALU 算出的地址，数据 RAM 写使能为 0。

2.3 精确异常

若只在写回级才清空流水线，SYSCALL 进入写回时，后面的 sw 已经到达执行级，数据 RAM 的写使能会在流水线被清空之前生效。因此异常指令一进入执行级就产生 ex_cancel。还在取指或译码、排在它后面的指令被清成无效，新的译码结果也不能进入执行级。ERET 同样拉高 ex_cancel，它后面的指令不会在返回之前执行。

以 syscall 后跟 sw 为例。syscall 在译码级时，sw 还在取指。下一拍 syscall 进入执行级，ex_cancel 为 1，sw 停在译码级且不能前进，这一拍结束时译码和取指的有效位被清掉。syscall 再经访存进入写回，这一拍更新 EPC、ExcCode 和 EXL，并清空流水线。sw 没有到过执行级，写使能也没有打开。处理程序返回后，这条 sw 会重新取指再执行，只是不能先于处理程序写回。

清空流水线时的目标地址由写回级给出：ERET 用当前 EPC，其它异常和中断用入口地址。指令 RAM 的数据晚一拍返回。清空的这一拍就把这个地址送到 inst_sram_addr，同时把取指 PC 设成同一地址、有效位置 1。下一拍读回的指令和这个 PC 配对，第一条进入译码的就是入口上的 mfhi。

写回级清空流水线时还会清掉执行级和访存级。中断在写回级才决定，被打断的指令后面的 store 可能正停在执行级；写使能同时被 wb_flush 掩掉，这一拍不会写入数据 RAM。执行级里尚未写完的乘除在这时丢掉进行中的状态，HI/LO 保持原值。

2.4 延迟槽

译码级当前指令是分支或跳转（不论是否发生转移）时，取指级正在取的下一条就是延迟槽。取指总线的最高位带上这个标记，该指令进入译码后成为它的 BD，再随流水线到达写回。延迟槽中的异常写入 CP0 时，EPC 写成 BD 所指示的分支 PC。

通用处理程序读 Cause 的位 31。该位为 0 时把 EPC 加 4，否则再加 4，然后 MTC0 写回并 ERET。普通异常因此返回到发生异常那条指令的下一条；延迟槽中的异常返回到分支后面的第二条，槽内指令不会再次执行。n83 到 n94 把 SYSCALL、BREAK、溢出等放进 BEQ 到 JALR 的延迟槽，检查的就是这条 EPC。

2.5 中断

中断请求为

```
int_req = IE && !EXL && !ERL && |(Status[15:8] & IP[7:0])
```

Status 位 15～8 是 IM7～IM0，与 Cause 的 IP7～IP0 按位对应。请求在写回级采样。当前指令自身已是异常时，优先把该异常写入 CP0；否则在 int_req 为 1 时，把这条正在写回的指令当作被中断打断的指令，ExcCode 记 0。这条指令的通用寄存器写回仍然完成，EPC 指向它。EXL 在同一拍置 1，下一拍起 int_req 为 0，处理程序执行期间不再采样新的中断。

Count 每拍翻转一位使能，使能为 1 的那一拍才加 1，和参考 CPU 的 count_add_en 相同。MTC0 写 Count 的那一拍用写入值代替自增。Count 与 Compare 相等时 TI 置 1；MTC0 写 Compare 的那一拍把 TI 清 0，并更新 Compare。

IP7～IP2 比中断源晚一拍写入 Cause。IP7 是外部硬件中断 5 与 TI 相或，IP6～IP2 分别是硬件中断 4～0。IP1、IP0 只由 MTC0 写 Cause 的位 9、位 8。SoC 把 hw_int[5:0] 接成 0，因此 func_lab9 里 IP7 只由定时器抬起。定时器测试把 IE 和全部 IM 打开，把 Count 设到 Compare 附近，然后执行 1: b 1b; nop。软件中断测试用同样的 Status，再 MTC0 把 IP1 或 IP0 写成 1。

中断处理程序要求 EPC 等于标号 1，然后自己把 EPC 加 8 再 ERET，从而跳过这条分支和它的延迟槽。被打断的是这条分支时，BD 为 0，EPC 就是分支 PC。被打断的是槽里的 nop 时，BD 为 1，EPC 仍是分支 PC。两条路径的返回地址相同。处理程序在读 Cause 时关闭 trace 比较，BD 的差别不会进入 golden_trace.txt。

2.6 MFC0 的数据相关

MFC0 的目的寄存器是 rt，数值要等写回级按 rd 读 CP0 才存在。执行级和访存级里的 ALU 结果不能前递。要用这个结果的指令还在译码级、而 MFC0 还在执行级或访存级时，按 load-use 的条件把 ds_ready_go 拉低，译码级停两拍。MFC0 到达写回后，rf_wdata 换成 CP0 读数，实验 1 的写回前递把这个数送给下一条。入口处理程序的典型序列是 mfc0 k0 紧接 bne k0, ...，比较用的就是这一路前递。

3 实验过程

3.1 程序与轨迹

func_lab8、func_lab9 放在 CPU_CDE/soft/，与 func_lab7 同级。两个工程里的 inst_ram 都是锁定的旧 IP，先 upgrade_ip，再把 COE 指到对应的 obj/inst_ram.coe 并重新生成仿真文件。cpu132_gettrace 跑完得到 golden_trace.txt，mycpu 的 testbench 在写非 0 寄存器时逐条比较。

func_lab8 的测试数为 $23 + 19 + 26 + 1 = 69$，新增的一点是 SYSCALL。func_lab9 再加 25 点，共 94。新增点依次是 Break，ADD/ADDI/SUB 溢出，LW/LH/LHU 读地址错，SW/SH 写地址错，取指地址错，保留指令，定时器中断，软件中断，以及放在 BEQ 到 JALR 延迟槽里的异常。

3.2 func_lab8

69 个功能点一次通过。第 42 点 mycpu 为 1077305 ns，gettrace 为 1193795 ns。第 68 点分别为 1644675 ns 和 1835425 ns。这两个时刻与实验 2

的 func_lab6、func_lab7 相同。

第 69 点是新增的 SYSCALL。mycpu 的通过时刻是 1649965 ns。gettrace 的通过时刻是 1841065 ns。结束时间见第 5.1 节。

3.3 func_lab9

换成 func_lab9 的 COE 后，前 69 点的通过时刻与 func_lab8 相同。第 70 到 94 点覆盖其余异常和中断，也全部通过。第 94 点 mycpu 为 1939805 ns，gettrace 为 2130615 ns。从第 69 点到第 94 点，两边各增加约 0.290 ms。

4 关键代码

4.1 译码级

inst_known 是实验 2 全部指令再加上三条 CP0 指令、SYSCALL 和 BREAK 的或。取指地址错单独判定，排在保留指令之前。溢出检查先做成一位，随指令进入执行级。

```
wire inst_adel_if = |ds_pc[1:0];
wire inst_ri      = ~inst_known & ~inst_adel_if;
wire ds_ex        = inst_syscall | inst_break | inst_ri | inst_adel_if;
wire [4:0] ds_excode = inst_adel_if ? `EXC_ADEL :
                       inst_syscall ? `EXC_SYS  :
                       inst_break   ? `EXC_BP   :
                       inst_ri      ? `EXC_RI   : 5'd0;
wire ds_ov_en   = inst_add | inst_addi | inst_sub;
wire ds_badv_we = inst_adel_if;
```

ds_ex 为 1 时，送到执行级的访存操作和乘除操作都改成空操作。保留指令的功能码即使碰巧等于 DIV，也不会把执行级停在除法器上。

4.2 执行级

es_alu_op[1] 为 1 表示减法。译码级已经判定的异常放在 es_ex 里，溢出和访存地址错只在它为 0 时生效，最后合成 es_ex_f。

```
wire es_ov = es_ov_en && (es_alu_op[1]
             ? (es_alu_src1[31] != es_alu_src2[31])
               && (alu_result[31] != es_alu_src1[31])
             : (es_alu_src1[31] == es_alu_src2[31])
               && (alu_result[31] != es_alu_src1[31]));
wire es_ex_f = es_ex || es_ov || es_adel_m || es_ades_m;
assign ex_cancel = es_valid && (es_ex_f || es_cp0_op == `CP0_ERET);
assign data_sram_wen =
    (es_valid && !wb_flush && !es_ex_f) ? st_wen : 4'h0;
```

es_adel_m、es_ades_m 按上一节的对齐规则产生。它们把 BadVAddr 改成 alu_result；取指地址错则继续使用译码级传来的 PC。

4.3 写入 CP0

下面几段在同一个时钟沿上赋值，异常、ERET、MTC0 的优先级写在 if / else if 里。IP7∼IP2 每拍都用上一拍的 TI 和 hw_int 更新。

```
cause_hwip <= {hw_int[5] | cause_ti, hw_int[4:0]};

if (ws_valid && ws_mtc0 && ws_addr == 5'd11)
    cause_ti <= 1'b0;
else if (count == compare)
    cause_ti <= 1'b1;

if (ws_valid && ws_ex && !status_exl)
    cause_epc <= ws_bd ? (ws_pc - 32'd4) : ws_pc;

if (ws_valid && ws_ex)
    status[1] <= 1'b1;
else if (ws_valid && ws_eret)
    status[1] <= status_erl ? status[1] : 1'b0;
else if (ws_valid && ws_mtc0 && ws_addr == 5'd12)
    status[1] <= ws_wdata[1];
```

4.4 清空流水线与 MFC0

清空流水线的这一拍，取指地址和取指 PC 使用同一个 wb_flush_pc。MFC0 在执行级或访存级、且目的寄存器等于译码级要用的 rs 或 rt 时停住译码，写回级再把 CP0 读数送进寄存器堆和前递。

```
assign wb_flush    = do_ex || ws_eret;
assign wb_flush_pc = ws_eret ? cp0_epc : ex_entry;
assign inst_sram_addr = wb_flush ? wb_flush_pc : nextpc;

wire mfc_use =
    (es_mfc && (es_dest != 5'd0) &&
     ((rs_need && es_dest == rs) || (rt_need && es_dest == rt)))
 || (ms_mfc && (ms_dest != 5'd0) &&
     ((rs_need && ms_dest == rs) || (rt_need && ms_dest == rt)));
assign ds_ready_go = ~load_use && ~mfc_use;
assign rf_we    = ws_gr_we && ws_valid && !take_ex;
assign rf_wdata = ws_mfc0 ? cp0_rdata : ws_final_result;
```

take_ex 只覆盖指令自身的异常。被中断打断的指令，rf_we 保持原值。EPC 仍指向该指令；中断处理程序再把 EPC 加 8，返回时跳过它。

5 实验结果

5.1 功能点对比

仿真控制台每隔约 10 µs 打印一行 `Test is running`。下面保留功能点通过行和结束行。左栏是 `cpu132_gettrace`，右栏是 `mycpu`。`gettrace` 结束时打印 `gettrace end!`；`mycpu` 结束时打印 `----PASS!!!`。

`func_lab8` 两边都到第 69 点。`gettrace` 在 1841525 ns 结束，`mycpu` 在 1650425 ns 结束。前 68 点与实验 2 的 `func_lab7` 时刻相同。

gettrace

```
Test begin!
----[  14055 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41545 ns] Number 8'd02 Functional Test Point PASS!!!
----[  63015 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90505 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117995 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145485 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172975 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200465 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227955 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255445 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276915 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 298385 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319855 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 333695 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 348585 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 420255 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 496875 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 559275 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 613675 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 615745 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 617815 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 619425 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 621035 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 643935 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 665405 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 687855 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 709325 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 730795 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 752265 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 773735 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 795205 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 822695 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 850185 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 877675 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 935355 ns] Number 8'd35 Functional Test Point PASS!!!
----[1019275 ns] Number 8'd36 Functional Test Point PASS!!!
----[1093745 ns] Number 8'd37 Functional Test Point PASS!!!
----[1160535 ns] Number 8'd38 Functional Test Point PASS!!!
----[1170115 ns] Number 8'd39 Functional Test Point PASS!!!
----[1179675 ns] Number 8'd40 Functional Test Point PASS!!!
----[1188615 ns] Number 8'd41 Functional Test Point PASS!!!
----[1193795 ns] Number 8'd42 Functional Test Point PASS!!!
----[1222025 ns] Number 8'd43 Functional Test Point PASS!!!
----[1265245 ns] Number 8'd44 Functional Test Point PASS!!!
----[1308185 ns] Number 8'd45 Functional Test Point PASS!!!
----[1351405 ns] Number 8'd46 Functional Test Point PASS!!!
----[1385005 ns] Number 8'd47 Functional Test Point PASS!!!
----[1432805 ns] Number 8'd48 Functional Test Point PASS!!!
----[1482775 ns] Number 8'd49 Functional Test Point PASS!!!
----[1537775 ns] Number 8'd50 Functional Test Point PASS!!!
----[1539745 ns] Number 8'd51 Functional Test Point PASS!!!
----[1547845 ns] Number 8'd52 Functional Test Point PASS!!!
----[1556585 ns] Number 8'd53 Functional Test Point PASS!!!
----[1565005 ns] Number 8'd54 Functional Test Point PASS!!!
----[1573745 ns] Number 8'd55 Functional Test Point PASS!!!
----[1583125 ns] Number 8'd56 Functional Test Point PASS!!!
----[1591865 ns] Number 8'd57 Functional Test Point PASS!!!
----[1598255 ns] Number 8'd58 Functional Test Point PASS!!!
----[1627845 ns] Number 8'd59 Functional Test Point PASS!!!
----[1659605 ns] Number 8'd60 Functional Test Point PASS!!!
----[1682825 ns] Number 8'd61 Functional Test Point PASS!!!
----[1709545 ns] Number 8'd62 Functional Test Point PASS!!!
----[1724385 ns] Number 8'd63 Functional Test Point PASS!!!
----[1740065 ns] Number 8'd64 Functional Test Point PASS!!!
----[1771905 ns] Number 8'd65 Functional Test Point PASS!!!
----[1801825 ns] Number 8'd66 Functional Test Point PASS!!!
----[1818865 ns] Number 8'd67 Functional Test Point PASS!!!
----[1835425 ns] Number 8'd68 Functional Test Point PASS!!!
----[1841065 ns] Number 8'd69 Functional Test Point PASS!!!
============================================================
gettrace end!
----Succeed in generating trace file!
$finish called at time : 1841525 ns : File "D:/Architecture/lab1/」
↪  lab3/CPU_CDE/cpu132_gettrace/testbench/tb_top.v" Line 221
```

mycpu

```
Test begin!
----[  14045 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41495 ns] Number 8'd02 Functional Test Point PASS!!!
----[  62925 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90375 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117825 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145275 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172725 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200175 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227625 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255075 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276505 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 297935 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319365 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 330605 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 342695 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 407295 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 475855 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 532215 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 580575 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 582455 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 584335 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 585745 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 587165 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 610025 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 631455 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 653865 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 675295 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 696725 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 718155 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 739585 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 761015 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 788465 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 815915 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 843365 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 905885 ns] Number 8'd35 Functional Test Point PASS!!!
----[ 993325 ns] Number 8'd36 Functional Test Point PASS!!!
----[1020055 ns] Number 8'd37 Functional Test Point PASS!!!
----[1044205 ns] Number 8'd38 Functional Test Point PASS!!!
----[1053745 ns] Number 8'd39 Functional Test Point PASS!!!
----[1063265 ns] Number 8'd40 Functional Test Point PASS!!!
----[1072165 ns] Number 8'd41 Functional Test Point PASS!!!
----[1077305 ns] Number 8'd42 Functional Test Point PASS!!!
----[1101865 ns] Number 8'd43 Functional Test Point PASS!!!
----[1140075 ns] Number 8'd44 Functional Test Point PASS!!!
----[1178045 ns] Number 8'd45 Functional Test Point PASS!!!
----[1216255 ns] Number 8'd46 Functional Test Point PASS!!!
----[1245945 ns] Number 8'd47 Functional Test Point PASS!!!
----[1290205 ns] Number 8'd48 Functional Test Point PASS!!!
----[1336495 ns] Number 8'd49 Functional Test Point PASS!!!
----[1387255 ns] Number 8'd50 Functional Test Point PASS!!!
----[1388925 ns] Number 8'd51 Functional Test Point PASS!!!
----[1395085 ns] Number 8'd52 Functional Test Point PASS!!!
----[1401885 ns] Number 8'd53 Functional Test Point PASS!!!
----[1408365 ns] Number 8'd54 Functional Test Point PASS!!!
----[1415165 ns] Number 8'd55 Functional Test Point PASS!!!
----[1422595 ns] Number 8'd56 Functional Test Point PASS!!!
----[1429385 ns] Number 8'd57 Functional Test Point PASS!!!
----[1434365 ns] Number 8'd58 Functional Test Point PASS!!!
----[1460075 ns] Number 8'd59 Functional Test Point PASS!!!
----[1487665 ns] Number 8'd60 Functional Test Point PASS!!!
----[1507925 ns] Number 8'd61 Functional Test Point PASS!!!
----[1531075 ns] Number 8'd62 Functional Test Point PASS!!!
----[1544165 ns] Number 8'd63 Functional Test Point PASS!!!
----[1558035 ns] Number 8'd64 Functional Test Point PASS!!!
----[1587035 ns] Number 8'd65 Functional Test Point PASS!!!
----[1614255 ns] Number 8'd66 Functional Test Point PASS!!!
----[1629675 ns] Number 8'd67 Functional Test Point PASS!!!
----[1644675 ns] Number 8'd68 Functional Test Point PASS!!!
----[1649965 ns] Number 8'd69 Functional Test Point PASS!!!
============================================================
Test end!
----PASS!!!
$finish called at time : 1650425 ns : File "D:/Architecture/lab1/」
↪  lab3/CPU_CDE/mycpu_verify/testbench/mycpu_tb.v" Line 267
```

`func_lab9` 两边都到第 94 点。`gettrace` 在 2131085 ns 结束，`mycpu` 在 1940275 ns 结束。第 69 点仍是 1841065 ns 与 1649965 ns。从第 42 点起两边相

差约 0.116 ms，到第 68 点及以后约为 0.191 ms。这是实验 2 里乘法指令拉开的差距，异常和中断没有再把它扩大。记录在第 69 点处分成两段，前一段与 `func_lab8` 相同。

gettrace

```
Test begin!
----[  14055 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41545 ns] Number 8'd02 Functional Test Point PASS!!!
----[  63015 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90505 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117995 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145485 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172975 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200465 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227955 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255445 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276915 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 298385 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319855 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 333695 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 348585 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 420255 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 496875 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 559275 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 613675 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 615745 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 617815 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 619425 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 621035 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 643935 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 665405 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 687855 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 709325 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 730795 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 752265 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 773735 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 795205 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 822695 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 850185 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 877675 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 935355 ns] Number 8'd35 Functional Test Point PASS!!!
----[1019275 ns] Number 8'd36 Functional Test Point PASS!!!
----[1093745 ns] Number 8'd37 Functional Test Point PASS!!!
----[1160535 ns] Number 8'd38 Functional Test Point PASS!!!
----[1170115 ns] Number 8'd39 Functional Test Point PASS!!!
----[1179675 ns] Number 8'd40 Functional Test Point PASS!!!
----[1188615 ns] Number 8'd41 Functional Test Point PASS!!!
----[1193795 ns] Number 8'd42 Functional Test Point PASS!!!
----[1222025 ns] Number 8'd43 Functional Test Point PASS!!!
----[1265245 ns] Number 8'd44 Functional Test Point PASS!!!
----[1308185 ns] Number 8'd45 Functional Test Point PASS!!!
----[1351405 ns] Number 8'd46 Functional Test Point PASS!!!
----[1385005 ns] Number 8'd47 Functional Test Point PASS!!!
----[1432805 ns] Number 8'd48 Functional Test Point PASS!!!
----[1482775 ns] Number 8'd49 Functional Test Point PASS!!!
----[1537775 ns] Number 8'd50 Functional Test Point PASS!!!
----[1539745 ns] Number 8'd51 Functional Test Point PASS!!!
----[1547845 ns] Number 8'd52 Functional Test Point PASS!!!
----[1556585 ns] Number 8'd53 Functional Test Point PASS!!!
----[1565005 ns] Number 8'd54 Functional Test Point PASS!!!
----[1573745 ns] Number 8'd55 Functional Test Point PASS!!!
----[1583125 ns] Number 8'd56 Functional Test Point PASS!!!
----[1591865 ns] Number 8'd57 Functional Test Point PASS!!!
----[1598255 ns] Number 8'd58 Functional Test Point PASS!!!
----[1627845 ns] Number 8'd59 Functional Test Point PASS!!!
----[1659605 ns] Number 8'd60 Functional Test Point PASS!!!
----[1682825 ns] Number 8'd61 Functional Test Point PASS!!!
----[1709545 ns] Number 8'd62 Functional Test Point PASS!!!
----[1724385 ns] Number 8'd63 Functional Test Point PASS!!!
----[1740065 ns] Number 8'd64 Functional Test Point PASS!!!
----[1771905 ns] Number 8'd65 Functional Test Point PASS!!!
----[1801825 ns] Number 8'd66 Functional Test Point PASS!!!
----[1818865 ns] Number 8'd67 Functional Test Point PASS!!!
----[1835425 ns] Number 8'd68 Functional Test Point PASS!!!
----[1841065 ns] Number 8'd69 Functional Test Point PASS!!!
```

mycpu

```
Test begin!
----[  14045 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41495 ns] Number 8'd02 Functional Test Point PASS!!!
----[  62925 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90375 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117825 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145275 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172725 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200175 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227625 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255075 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276505 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 297935 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319365 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 330605 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 342695 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 407295 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 475855 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 532215 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 580575 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 582455 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 584335 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 585745 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 587165 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 610025 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 631455 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 653865 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 675295 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 696725 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 718155 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 739585 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 761015 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 788465 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 815915 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 843365 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 905885 ns] Number 8'd35 Functional Test Point PASS!!!
----[ 993325 ns] Number 8'd36 Functional Test Point PASS!!!
----[1020055 ns] Number 8'd37 Functional Test Point PASS!!!
----[1044205 ns] Number 8'd38 Functional Test Point PASS!!!
----[1053745 ns] Number 8'd39 Functional Test Point PASS!!!
----[1063265 ns] Number 8'd40 Functional Test Point PASS!!!
----[1072165 ns] Number 8'd41 Functional Test Point PASS!!!
----[1077305 ns] Number 8'd42 Functional Test Point PASS!!!
----[1101865 ns] Number 8'd43 Functional Test Point PASS!!!
----[1140075 ns] Number 8'd44 Functional Test Point PASS!!!
----[1178045 ns] Number 8'd45 Functional Test Point PASS!!!
----[1216255 ns] Number 8'd46 Functional Test Point PASS!!!
----[1245945 ns] Number 8'd47 Functional Test Point PASS!!!
----[1290205 ns] Number 8'd48 Functional Test Point PASS!!!
----[1336495 ns] Number 8'd49 Functional Test Point PASS!!!
----[1387255 ns] Number 8'd50 Functional Test Point PASS!!!
----[1388925 ns] Number 8'd51 Functional Test Point PASS!!!
----[1395085 ns] Number 8'd52 Functional Test Point PASS!!!
----[1401885 ns] Number 8'd53 Functional Test Point PASS!!!
----[1408365 ns] Number 8'd54 Functional Test Point PASS!!!
----[1415165 ns] Number 8'd55 Functional Test Point PASS!!!
----[1422595 ns] Number 8'd56 Functional Test Point PASS!!!
----[1429385 ns] Number 8'd57 Functional Test Point PASS!!!
----[1434365 ns] Number 8'd58 Functional Test Point PASS!!!
----[1460075 ns] Number 8'd59 Functional Test Point PASS!!!
----[1487665 ns] Number 8'd60 Functional Test Point PASS!!!
----[1507925 ns] Number 8'd61 Functional Test Point PASS!!!
----[1531075 ns] Number 8'd62 Functional Test Point PASS!!!
----[1544165 ns] Number 8'd63 Functional Test Point PASS!!!
----[1558035 ns] Number 8'd64 Functional Test Point PASS!!!
----[1587035 ns] Number 8'd65 Functional Test Point PASS!!!
----[1614255 ns] Number 8'd66 Functional Test Point PASS!!!
----[1629675 ns] Number 8'd67 Functional Test Point PASS!!!
----[1644675 ns] Number 8'd68 Functional Test Point PASS!!!
----[1649965 ns] Number 8'd69 Functional Test Point PASS!!!
```

gettrace

```
----[1846915 ns] Number 8'd70 Functional Test Point PASS!!!
----[1853605 ns] Number 8'd71 Functional Test Point PASS!!!
----[1860205 ns] Number 8'd72 Functional Test Point PASS!!!
----[1866895 ns] Number 8'd73 Functional Test Point PASS!!!
----[1874275 ns] Number 8'd74 Functional Test Point PASS!!!
----[1881675 ns] Number 8'd75 Functional Test Point PASS!!!
----[1889055 ns] Number 8'd76 Functional Test Point PASS!!!
----[1896725 ns] Number 8'd77 Functional Test Point PASS!!!
----[1904395 ns] Number 8'd78 Functional Test Point PASS!!!
----[1911965 ns] Number 8'd79 Functional Test Point PASS!!!
----[1918905 ns] Number 8'd80 Functional Test Point PASS!!!
----[1937725 ns] Number 8'd81 Functional Test Point PASS!!!
----[1946945 ns] Number 8'd82 Functional Test Point PASS!!!
----[1962245 ns] Number 8'd83 Functional Test Point PASS!!!
----[1977545 ns] Number 8'd84 Functional Test Point PASS!!!
----[1992845 ns] Number 8'd85 Functional Test Point PASS!!!
----[2008145 ns] Number 8'd86 Functional Test Point PASS!!!
----[2023445 ns] Number 8'd87 Functional Test Point PASS!!!
----[2038745 ns] Number 8'd88 Functional Test Point PASS!!!
----[2054055 ns] Number 8'd89 Functional Test Point PASS!!!
----[2069375 ns] Number 8'd90 Functional Test Point PASS!!!
----[2084685 ns] Number 8'd91 Functional Test Point PASS!!!
----[2099995 ns] Number 8'd92 Functional Test Point PASS!!!
----[2115305 ns] Number 8'd93 Functional Test Point PASS!!!
----[2130615 ns] Number 8'd94 Functional Test Point PASS!!!
================================================================
gettrace end!
----Succeed in generating trace file!
$finish called at time : 2131085 ns : File "D:/Architecture/lab1/
↪  lab3/CPU_CDE/cpu132_gettrace/testbench/tb_top.v" Line 221
```

mycpu

```
----[1655465 ns] Number 8'd70 Functional Test Point PASS!!!
----[1661805 ns] Number 8'd71 Functional Test Point PASS!!!
----[1668055 ns] Number 8'd72 Functional Test Point PASS!!!
----[1674395 ns] Number 8'd73 Functional Test Point PASS!!!
----[1681565 ns] Number 8'd74 Functional Test Point PASS!!!
----[1688755 ns] Number 8'd75 Functional Test Point PASS!!!
----[1695925 ns] Number 8'd76 Functional Test Point PASS!!!
----[1703385 ns] Number 8'd77 Functional Test Point PASS!!!
----[1710845 ns] Number 8'd78 Functional Test Point PASS!!!
----[1718215 ns] Number 8'd79 Functional Test Point PASS!!!
----[1724805 ns] Number 8'd80 Functional Test Point PASS!!!
----[1742565 ns] Number 8'd81 Functional Test Point PASS!!!
----[1750735 ns] Number 8'd82 Functional Test Point PASS!!!
----[1766485 ns] Number 8'd83 Functional Test Point PASS!!!
----[1782235 ns] Number 8'd84 Functional Test Point PASS!!!
----[1797985 ns] Number 8'd85 Functional Test Point PASS!!!
----[1813735 ns] Number 8'd86 Functional Test Point PASS!!!
----[1829485 ns] Number 8'd87 Functional Test Point PASS!!!
----[1845235 ns] Number 8'd88 Functional Test Point PASS!!!
----[1860995 ns] Number 8'd89 Functional Test Point PASS!!!
----[1876765 ns] Number 8'd90 Functional Test Point PASS!!!
----[1892525 ns] Number 8'd91 Functional Test Point PASS!!!
----[1908285 ns] Number 8'd92 Functional Test Point PASS!!!
----[1924045 ns] Number 8'd93 Functional Test Point PASS!!!
----[1939805 ns] Number 8'd94 Functional Test Point PASS!!!
================================================================
Test end!
----PASS!!!
$finish called at time : 1940275 ns : File "D:/Architecture/lab1/
↪  lab3/CPU_CDE/mycpu_verify/testbench/mycpu_tb.v" Line 267
```

程序	mycpu 结束	gettrace 结束
func_lab8（69 点）	1.650 ms	1.842 ms
func_lab9（94 点）	1.940 ms	2.131 ms

5.2 定向程序与波形

func_lab9 比较的是写回寄存器的数据。为了直接看到 Cause 和 EPC，lab3/sim/wave_tb.v 跑一段只含异常和中断的程序，每条事件在 0xbfc00380 的处理程序里把当时的 Cause、EPC 存到 0x80000100。异常则把 EPC 加 4 再 ERET，返回后写下序号。程序一开始把 Compare 写成全 1，复位后 Count 与 Compare 同为 0 时不会把 TI 留在 Cause 里。Status 先只开 BEV。五条异常都返回后，0x80000000 起的序号是 1、2、3、4、5。

事件	Cause	EPC	ExcCode
SYSCALL	0x00000020	0xbfc00028	8
BREAK	0x00000024	0xbfc00038	9
有符号加溢出	0x00000030	0xbfc0004c	12
取数地址错	0x00000010	0xbfc0005c	4
保留指令	0x00000028	0xbfc0006c	10
软件中断	0x00000100	0xbfc000a0	0
时钟中断	0xc0008000	0xbfc000c4	0
硬件中断	0x00000400	0xbfc000cc	0

处理程序存下的 Cause 按位域来读：位 31 是 BD，位 30 是 TI，位 15～8 是 IP7～IP0，位 6～2 是 ExcCode。五条异常的 BD、TI 和 IP 都是 0，Cause 就是 ExcCode 左移 2 位。8、9、12、4、10 分别得到 `0x20`、`0x24`、`0x30`、`0x10`、`0x28`，与表中五个数一致。异常写回的下一拍 Status 为 `0x00400002`，即原来的 BEV 再加上 EXL。`ERET` 的下一拍 EXL 清掉，Status 回到 `0x00400000`，EPC 已经加 4。

SYSCALL 的 EPC `0xbfc00028` 就是这条指令的 PC。第 15 拍它还在执行级，`ex_cancel` 为 1；第 16 拍译码级 valid 变 0；第 17 拍写回 PC 为 `0xbfc00028` 并清空流水线；第 18 拍取指地址已是 `0xbfc00380`，EPC 也已写成 `0xbfc00028`。紧跟着的 `sw $t0, 0x200($s0)` 在这之前没有 `data_wen`。处理程序把 EPC 加 4，`ERET` 回到 `0xbfc0002c`，第 37 拍这条 `sw` 才把 `0x1111` 写进 `0x80000200`。图里 `data_wen` 的脉冲都在进入处理程序之后。返回后序号 1 写到 `0x80000000`。

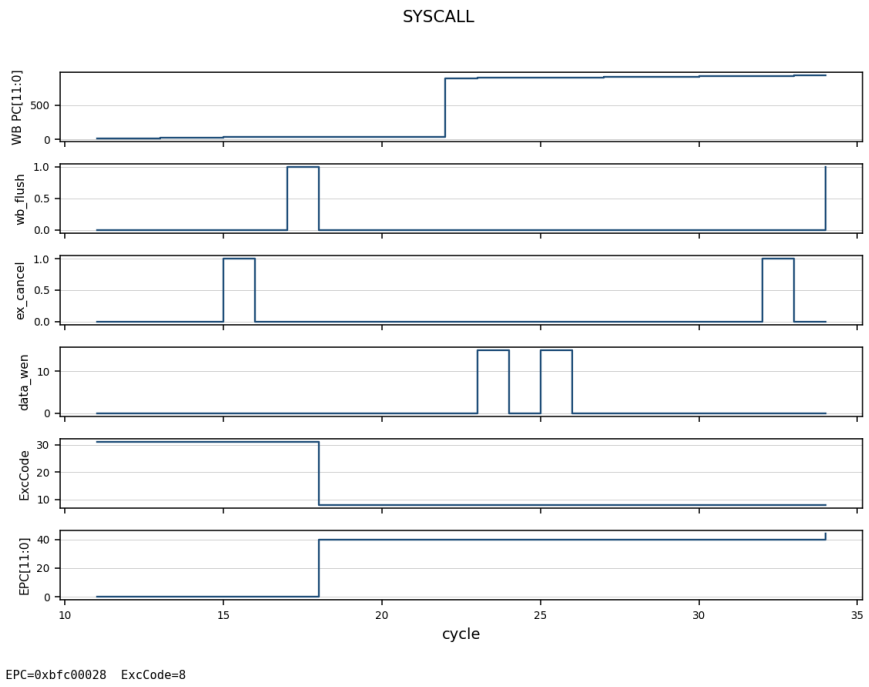


图 1: SYSCALL。ExcCode 在清空流水线的下一拍变为 8，EPC 低 12 位为 `0x028`。

溢出是 `add` 把两个 `0x80000000` 相加。第 68 拍写回 PC 和下一拍的 EPC 都是 `0xbfc0004c`，这条 `add` 的地址；ExcCode 为 12。BREAK、对地址 1 的 `lw`、字 `0xffffffff` 同样在写回当拍对准自己的 PC，ExcCode 为 9、4、10。五次返回后 `0x80000000` 起的序号是 1、2、3、4、5，说明 EPC 加 4 都落在异常指令的下一条，没有把异常指令再执行一遍。

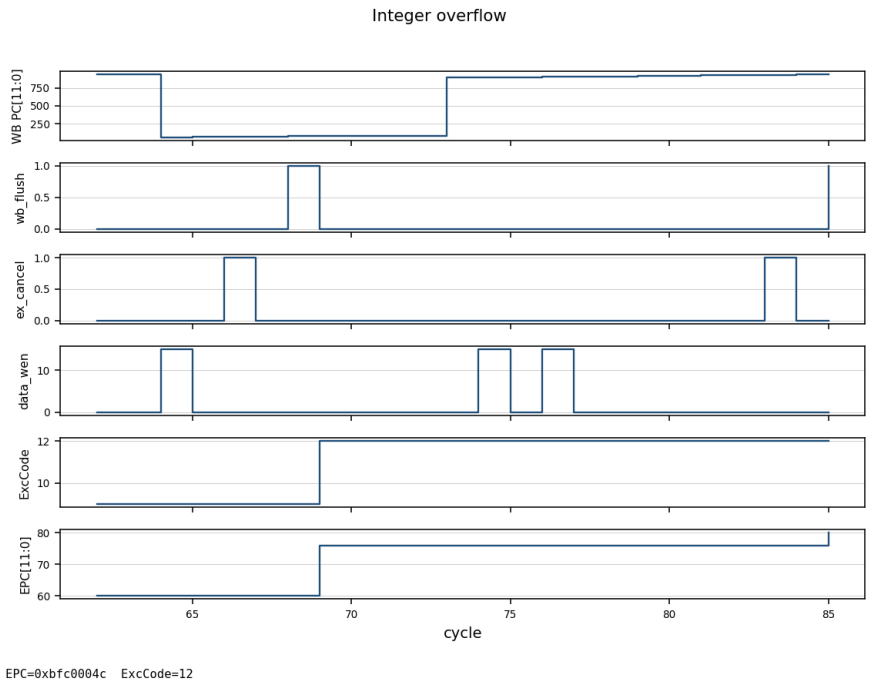


图 2: 整数溢出。ExcCode 变为 12，EPC 低 12 位为 `0x04c`。

5.3 三种中断

软件中断由 `MTC0` 把 Cause 的位 8 写成 1。Compare 仍是全 1，所以 TI 保持 0。IP0 要等这拍写入之后才参与 `int_req`，因此被打断的不是 `MTC0` 本身，而是它后面的第一条 `nop`。第 152 拍写回 PC 为 `0xbfc000a0`。下一拍 Cause 为 `0x00000100`，EPC 也是 `0xbfc000a0`，Status 为 `0x0040ff03`：BEV、八位 IM、IE 和 EXL 都为 1。`0x100` 只有 IP0，ExcCode 和 BD 都是 0。分支在 `0xbfc000a8`，这一拍还没有写回。处理程序清掉 IP0，并把 EPC 改到时钟中断的入口 `0xbfc000b0`。第 175 拍 `ERET` 之后 Cause 为 0，Status 为 `0x0040ff01`，IE 仍开着，EXL 已清。`func_lab9` 的 `n82` 把同样的请求留在 `b 1b` 上，轨迹里的 EPC 等于该分支。两条路径都是写回级采样，受害指令是当时正要写回的那一条。

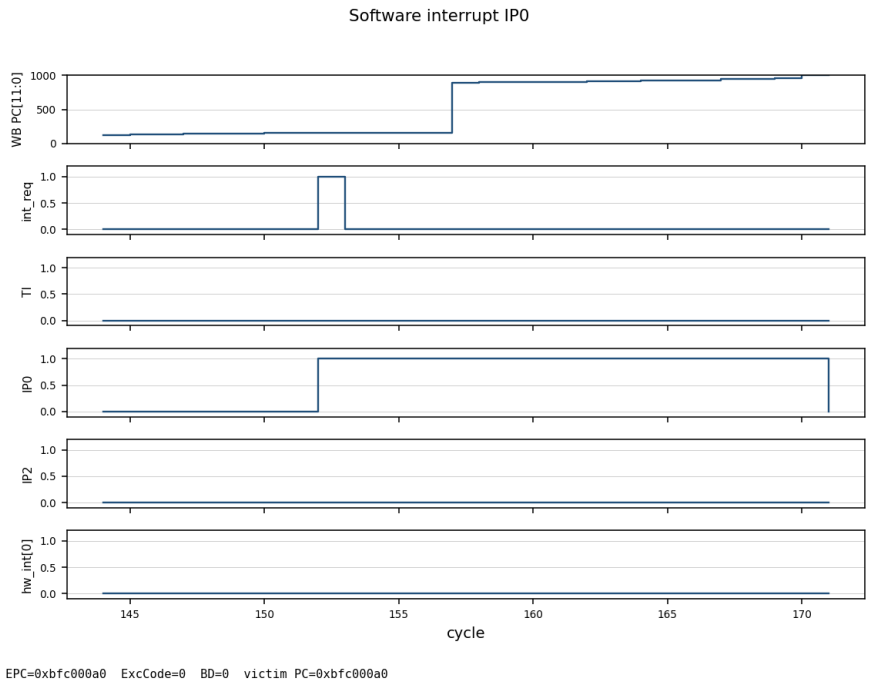


图 3: 软件中断。IP0 抬高后 `int_req` 出现一拍，写回 PC 转到 `0xbfc00380`。

时钟中断把 Count 清 0、Compare 写成 80，然后执行 `b 1b; nop`。TI 先变 1，IP7 和 `int_req` 晚一拍。第 342 拍写回的是 `0xbfc000c8`，即延迟槽里的 `nop`。下一拍 Cause 为 `0xc0008000`：位 31 的 BD 为 1，位 30 的 TI 为 1，IP7 为 1，ExcCode 为 0。BD 为 1 时 EPC 取受害指令的前一条，所以 EPC 是分支 `0xbfc000c4`，而不是槽内的 `nop`。Status 同时为 `0x0040ff03`。处理程序把 Compare 写成全 1 来清 TI。它 `ERET` 的下一拍 Cause 变成 `0x80000000`：BD 还留着，TI 和 IP7 已经是 0，EPC 被指到 `0xbfc000cc`。

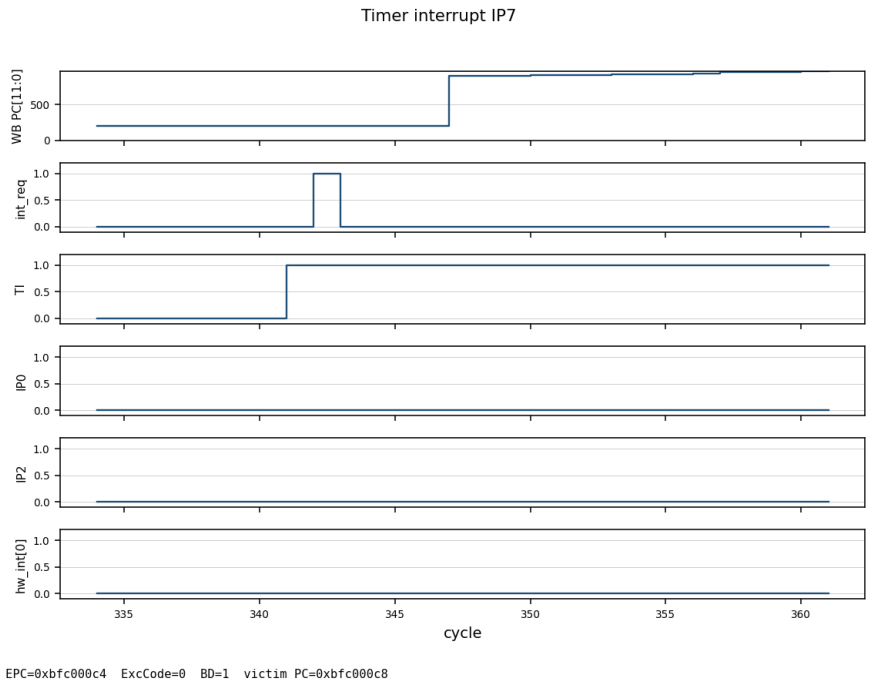


图 4: 时钟中断。TI 先于 `int_req` 变 1。受害指令在延迟槽，EPC 是前面的分支。

`soc_lite_top` 在功能测试里把 `hw_int[5:0]` 接成 0，`func_lab9` 因此不会从引脚上进入硬件中断。定向程序跑到 `0xbfc000cc` 的分支后，testbench 把 `hw_int[0]` 拉高。IP2 比引脚晚一拍，`int_req` 跟着 IP2。第 376 拍写回 PC 就是这条分支。下一拍 Cause 为 `0x00000400`，位 10 是 IP2，ExcCode 和 BD 都是 0，因此 EPC 等于受害指令本身 `0xbfc000cc`。Status 为 `0x0040ff03`。处理程序把 Status 写成 `0x00400000`，IE 关掉，EPC 写成结束循环 `0xbfc000d4`。`ERET` 之后停在这个循环，`0x80000014` 为 `0x55`。仿真打印 `----WAVE PASS!!!----`。

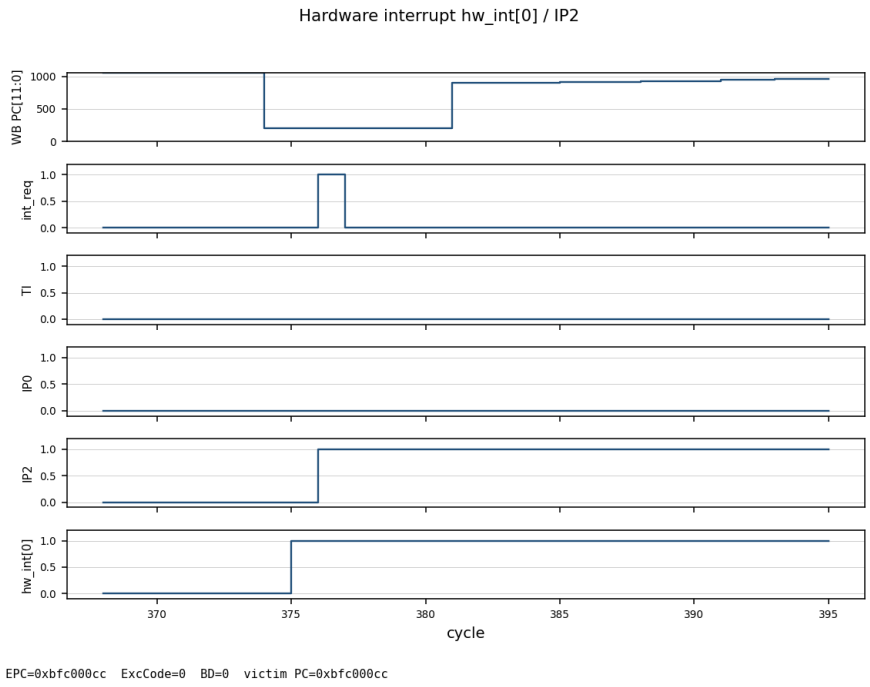


图 5: 硬件中断 0。`hw_int[0]` 拉高后 IP2 置 1，EPC 为分支 `0xbfc000cc`。

5.4 记忆游戏

记忆游戏的源程序在 `lab3/lab9/memory_game`。`memory_game()` 先把灯写成 `0xffff`，用 confreg 计时器做随机种子，然后反复做四件事：等一次按键，用 `my_rand` 产生 8 个图案，把图案显示出来，再读 8 次按键并打分。每个图案是 1、2、4、8 中的一个。显示时 `SOC_LED` 写成 `~pattern[i]`，所以图案 8、4、2、1 对应的灯是 `0xfff7`、`0xfffb`、`0xfffd`、`0xfffe`。读键时 `getUserPress` 取 `SOC_BTN_KEY[15:12]` 再把这四位反过来。扫描到的键码 `0x1000`、`0x2000`、`0x4000`、`0x8000` 因此读回来是 8、4、2、1。每采样到一次按键，程序立刻把灯写成 `~pb`，松开后再写回 `0xffff`。`displayScore` 数 `pattern[i]==keys[i]` 的个数。`SOC_NUM` 的高字节是已经猜过的次数，低字节是得分；八次全对时 `SOC_LED_RG1` 写 1，否则写 2，`SOC_LED_RG0` 写 1。

仿真时钟和计时器时钟都是 100 MHz，`CPU_COUNT_PER_US` 是 100。按源程序的毫秒延时，一轮要十几秒仿真时间。`confreg` 只在定义了 `SIM_FAST_IO` 时把计时器每次加 5000，并把按键去抖改到 `key_count[5]`。游戏二进制没有改。`SIM_FAST_IO` 只由记忆游戏的仿真脚本定义，`func_lab9` 的计时器仍是每拍加 1。数据 RAM 从 0 开始。`my_srand` 在出题前把种子写好，图案和按键都放在栈上。

testbench 先按一次键让游戏进入出题，再按灯的跳变记下 8 个图案，然后用同样的顺序回放。目标列被扫描到时把按键行拉成 `0111`，其余列保持松开。程序按刚读到的键把灯点亮后，testbench 核对这盏灯就是刚才那个图案，再松

开，等灯回到 0xffff 后按下一次。

这一轮的图案是 8、8、1、2、8、2、8、2。八次按键点亮的灯是 fff7、fff7、fffe、fffd、fff7、fffd、fff7、fffd。低四位里被点亮的那一位正好是图案 8、8、1、2、8、2、8、2，出题的灯和按键点亮的灯逐格相同。



图 6: 八步里出题时的灯和按键点亮的灯。点亮的圆点是 SOC_LED 低四位中为 0 的那一位。

detectKeyPresses 每读到一次就把 SOC_NUM 写成猜测次数左移 24 位，八次之后是 0x08000000。displayScore 再按位或上得分。八格都相同，得分是 8，结果为 0x08000008。led_rg0 为 1。得分等于 8 时 led_rg1 为 1。仿真打印 ----GAME PASS!!!----。

出题从第 2 盏灯起，相邻两次亮灯间隔都是 16148 拍。delay(800) 在计时器每拍加 5000 时要等 $800 \times 100 \times 100/5000 = 16000$ 拍，多出的 148 拍是写灯和循环本身。第一段间隔是 8132 拍，接近 delay(400) 的 8000 拍。delay 先写 SOC_TIMER=0，清零脉冲比紧跟着的第一次读晚几拍。这一次读到的若仍是上次留下的大数，循环马上结束，清零要到下一次 delay 才起作用。因此 delay(800) 和 delay(400) 交替成为真正的等待，亮灯间隔就在 16148 拍和 8132 拍之间换。回放里也能看到同一现象：按下到灯按这个键点亮是 76～112 拍，这是去抖后的列扫描；灯点亮到下一次按下则在 8076 拍和 60 拍之间交替，8076 拍就是一次完整的 delay(400)。间隔变短没有改掉读到的键，八次按键点亮的灯仍与出题一致。

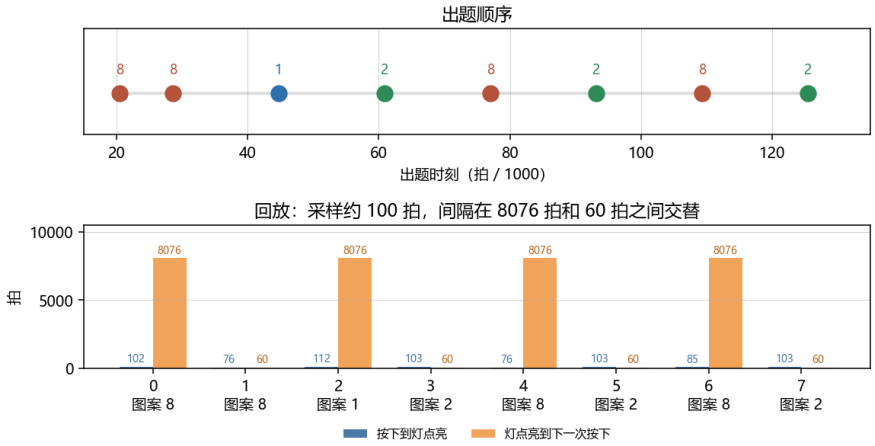


图 7: 出题时刻，以及回放时按下到灯点亮、灯点亮到下一次按下的拍数。

实验箱未接，按键和灯都走 confreg 的仿真模型，没有下载比特流。

6 实验收获

异常指令一进入执行级，就清掉排在它后面的指令，写回级才把 EXL、EPC 和 ExcCode 写入 CP0。如果等到写回级才清空流水线，`SYSCALL` 后面的 `sw` 会先写进数据 RAM。`MFC0` 比 ALU 晚两级才有结果，入口处 `mfc0` 紧挨 `bne` 时要按 load-use 停住译码，写回级再把 CP0 的读数前递出去。Cause 和 Status 的位域要和参考 CPU 一致：trace 比较的是 `MFC0` 的整字，多出来的 TI 或 IP 会在比较中直接报错。Count 隔拍加 1、IP7 比 TI 晚一拍之后，定时器和软件中断都发生在测试程序的原地循环 `b 1b` 上。EPC 等于该分支的 PC，处理程序加 8 后同时跳过这条分支和它的延迟槽。定向程序里，时钟中断打在延迟槽的 `nop` 上，BD 为 1，EPC 仍是前面的分支；硬件中断 0 打在分支本身上，IP2 比 `hw_int[0]` 晚一拍。记忆游戏在仿真里走完一轮：八次按键点亮的灯与出题的灯一致，数码管为 `0x08000008`，右侧双色灯为 1。

7 附：源码目录

CPU 仍在实验 1 的工程里改。`lab3` 放测试程序、仿真脚本和本报告。

- `lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/cp0.v`
- `lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/IF_stage.v`
- `lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/ID_stage.v`
- `lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/EXE_stage.v`
- `lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/MEM_stage.v`
- `lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/WB_stage.v`
- `lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/mycpu.h`

- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/mycpu_top.v
- lab1/lab3/CPU_CDE/cpu132_gettrace/golden_trace.txt
- lab1/lab3/CPU_CDE/soft/func_lab8/obj/inst_ram.coe
- lab1/lab3/CPU_CDE/soft/func_lab9/obj/inst_ram.coe
- lab3/sim/run_gettrace_lab8.tcl
- lab3/sim/run_mycpu_lab8.tcl
- lab3/sim/run_gettrace_lab9.tcl
- lab3/sim/run_mycpu_lab9.tcl
- lab3/sim/wave_tb.v
- lab3/sim/gen_wave_imem.py
- lab3/sim/game_tb.v
- lab3/lab9/memory_game/memory_game/memory_main.c
- lab3/report/report.tex